ĐẶC TẢ YÊU CẦU PHẦN MỀM

CERTIFICATE VERIFICATION DAPP

(Hệ thống xác minh chứng chỉ số trên Blockchain)

1. GIỚI THIỆU (INTRODUCTION)

1.1. Mục đích (Purpose)

Tài liệu này mô tả đặc tả yêu cầu phần mềm (Software Requirements Specification – SRS) cho hệ thống: **Certificate Verification DApp**

Hệ thống được xây dựng nhằm cung cấp một nền tảng xác minh chứng chỉ số dựa trên công nghệ Blockchain, với các mục tiêu chính:

- Cho phép tổ chức (Issuer) phát hành chứng chỉ số
- Đảm bảo **tính bất biến (immutability)** thông qua Blockchain
- Cho phép bên thứ ba xác minh chứng chỉ nhanh chóng
- Ngăn chặn hành vi làm giả chứng chỉ

Tài liệu này được sử dụng để:

- Làm cơ sở thiết kế hệ thống
- Hướng dẫn phát triển và triển khai
- Hỗ trợ kiểm thử và đánh giá hệ thống
- Là tài liệu tham chiếu cho bảo trì và mở rộng

1.2. Phạm vi hệ thống (System Scope)

Hệ thống là một **Decentralized Application (DApp)** chạy trên nền web, kết hợp:

- **Off-chain (Database – SQL Server)**
- **On-chain (Blockchain – Smart Contract)**

Phân tách dữ liệu:

Thành phần	Lưu trữ
Database	Thông tin chi tiết chứng chỉ
Blockchain	Hash + trạng thái xác thực

Chức năng chính:

- Phát hành chứng chỉ (Issue)
- Xác minh chứng chỉ (Verify)
- Thu hồi chứng chỉ (Revoke)
- Xem chứng chỉ
- Xác minh qua QR Code

1.3 Đối tượng sử dụng tài liệu

Tài liệu này được sử dụng bởi:
- Nhóm phát triển hệ thống (Developer)
- Giảng viên hướng dẫn
- Người kiểm thử hệ thống (Tester)
- Người đánh giá đồ án

1.4. Định nghĩa thuật ngữ (Definitions)

Thuật ngữ	Ý nghĩa
DApp	Ứng dụng phi tập trung
Blockchain	Sổ cái phân tán
Hash	Giá trị băm dùng để xác minh dữ liệu
MetaMask	Ví Web3 dùng để ký transaction
Issuer	Tổ chức cấp chứng chỉ
Holder	Người sở hữu chứng chỉ
Verifier	Người xác minh
QR Code	Mã truy cập nhanh

2. MÔ TẢ TỔNG QUAN HỆ THỐNG

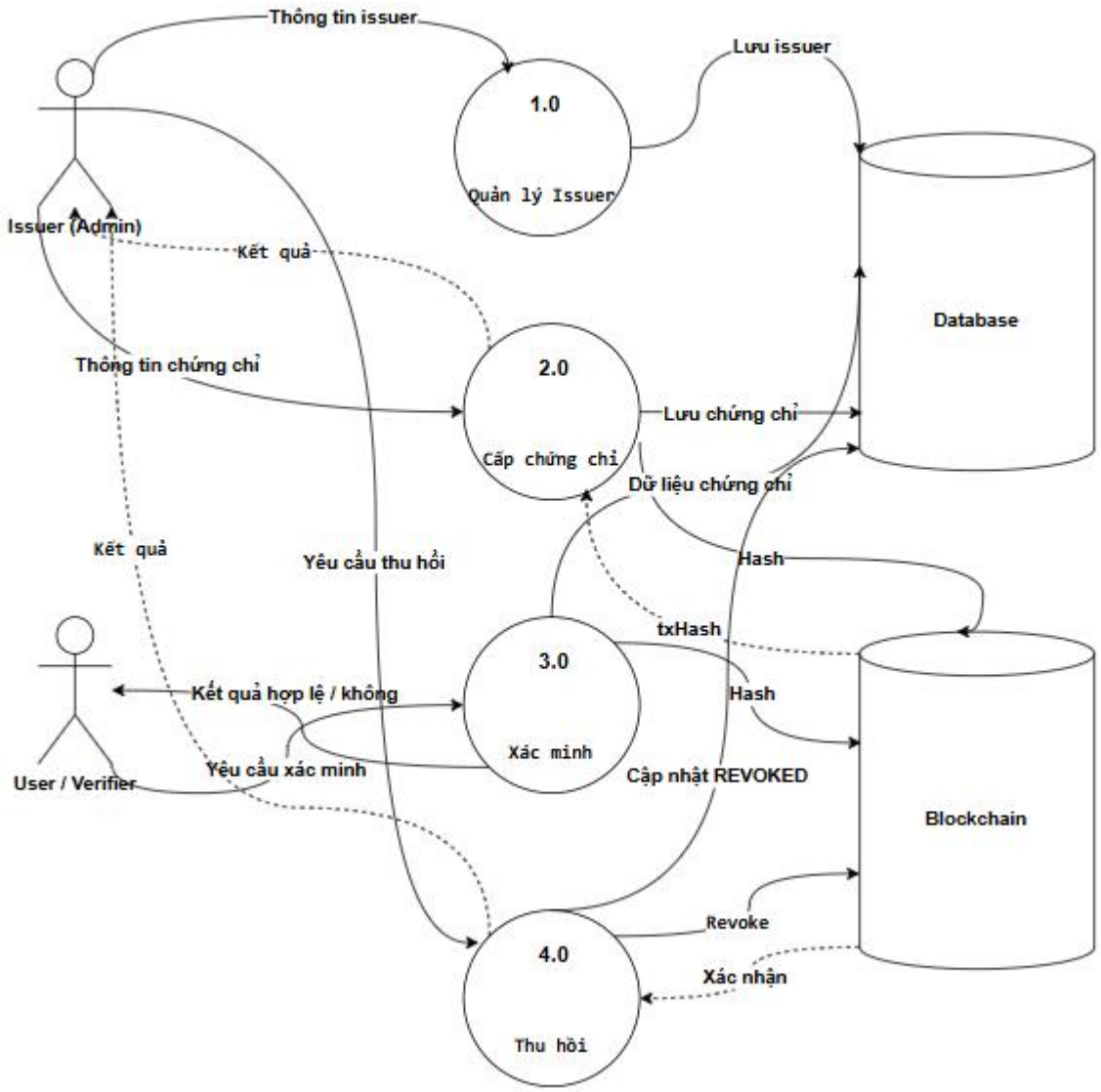
2.1. Góc nhìn sản phẩm (Product Perspective)

Hệ thống sử dụng kiến trúc:

Web Application + Blockchain (Hybrid DApp Architecture)

Các thành phần chính:

1. Frontend (React)
2. Backend (Node.js / Express)
3. Database (SQL Server)
4. Blockchain (Ethereum Smart Contract)
5. MetaMask (Wallet)



Product Perspective : mô tả kiến trúc hệ thống.

2.2. Kiến trúc trách nhiệm (Component Responsibilities)

Frontend

- Giao diện người dùng
- Gửi request đến Backend
- Kết nối MetaMask
- Gửi yêu cầu ký transaction

Backend

- Xử lý logic nghiệp vụ
- Tạo DataHash
- Lưu dữ liệu vào Database
- Đồng bộ trạng thái blockchain

Blockchain (Smart Contract)

- Lưu Hash chứng chỉ
- Kiểm tra tính hợp lệ
- Ghi nhận trạng thái revoke

Database

- Lưu toàn bộ dữ liệu chi tiết
- Lưu trạng thái chứng chỉ
- Lưu transaction hash

MetaMask

- Lưu Private Key của user
- Ký transaction
- Gửi transaction lên blockchain

2.3. Mô hình bảo mật (Security Model)

Nguyên tắc chuẩn:

Private Key KHÔNG được lưu trên Backend

Luồng ký transaction chuẩn:

Frontend → MetaMask → Blockchain

*Ghi chú triển khai thực tế

Trong hệ thống hiện tại:

- Backend có sử dụng PRIVATE_KEY trong .env
- Đây là **tài khoản hệ thống phục vụ DEMO**

Trong môi trường production, toàn bộ transaction phải được ký thông qua MetaMask nhằm đảm bảo tính phi tập trung.
Trong phạm vi demo, hệ thống sử dụng một tài khoản hệ thống (system wallet) để đơn giản hóa quá trình triển khai và kiểm thử.

2.3 Các loại người dùng (User Classes)

- Issuer:

Tổ chức có quyền phát hành chứng chỉ.

Có quyền tạo chứng chỉ và thu hồi chứng chỉ.

Student (Holder):

Người sở hữu chứng chỉ.

Có thể xem chứng chỉ của mình.

- Verifier:

Bên thứ ba kiểm tra tính hợp lệ của chứng chỉ.

Có thể xác minh bằng CertificateID hoặc QR Code.

2.5 Giả định và ràng buộc (Assumptions and Constraints)

- Người dùng phải có ví MetaMask.

- Blockchain sử dụng mạng thử nghiệm (Test Network(Sepolia) hoặc Ganache).

- Hệ thống phụ thuộc RPC Node

- Backend không được lưu Private Key của người dùng.

- Hệ thống phụ thuộc vào RPC Node để giao tiếp với Blockchain.

- RPC Node đóng vai trò trung gian giữa Backend/Frontend và mạng Blockchain.

- Nếu RPC Node không hoạt động, hệ thống sẽ không thể:

  + Gửi transaction lên Blockchain

  + Truy vấn dữ liệu on-chain

  + Thực hiện xác minh chứng chỉ

3. KIẾN TRÚC HỆ THỐNG (SYSTEM ARCHITECTURE)

3.1. Luồng xử lý tổng thể (Correct DApp Flow)

Kiến trúc của hệ thống tuân theo mô hình DApp chuẩn.

Luồng xử lý chính:

Frontend → Backend → Frontend → MetaMask → Blockchain → Frontend → Backend

Lưu ý:Blockchain không được truy cập trực tiếp mà thông qua RPC Endpoint

(ví dụ: Infura, Alchemy, hoặc Public RPC).

RPC Node đóng vai trò:

- Nhận transaction từ MetaMask hoặc Backend

- Truyền dữ liệu đến Blockchain

- Trả kết quả về hệ thống

3.2. Luồng Issue Certificate (CHI TIẾT)

Bước 1

User nhập dữ liệu trên Frontend

Bước 2

Frontend → Backend

Bước 3

Backend:

- Tạo DataHash
- Lưu DB (PENDING)

Bước 4

Frontend gọi MetaMask ký

Bước 5

MetaMask → Blockchain

Bước 6

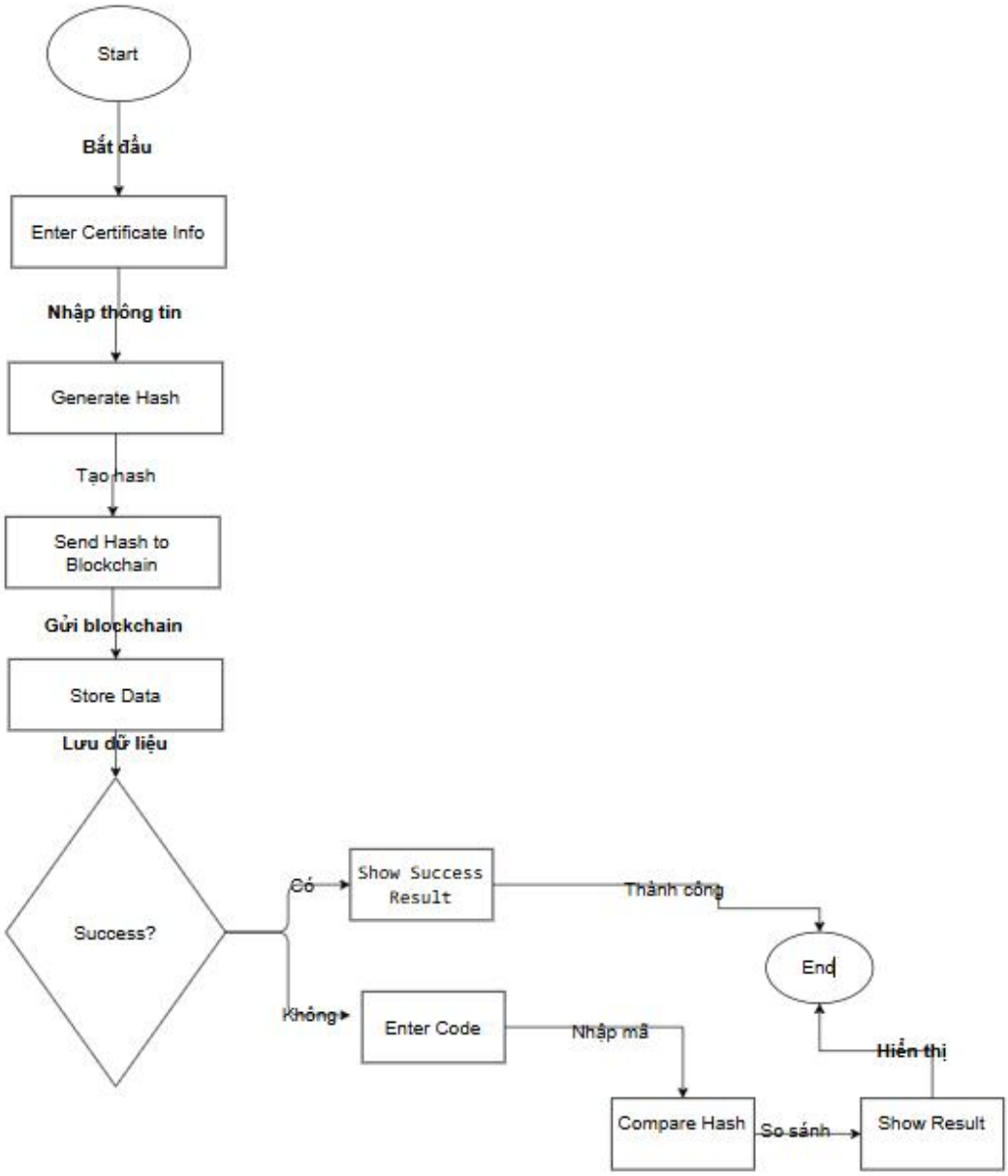
Blockchain trả txHash

Bước 7

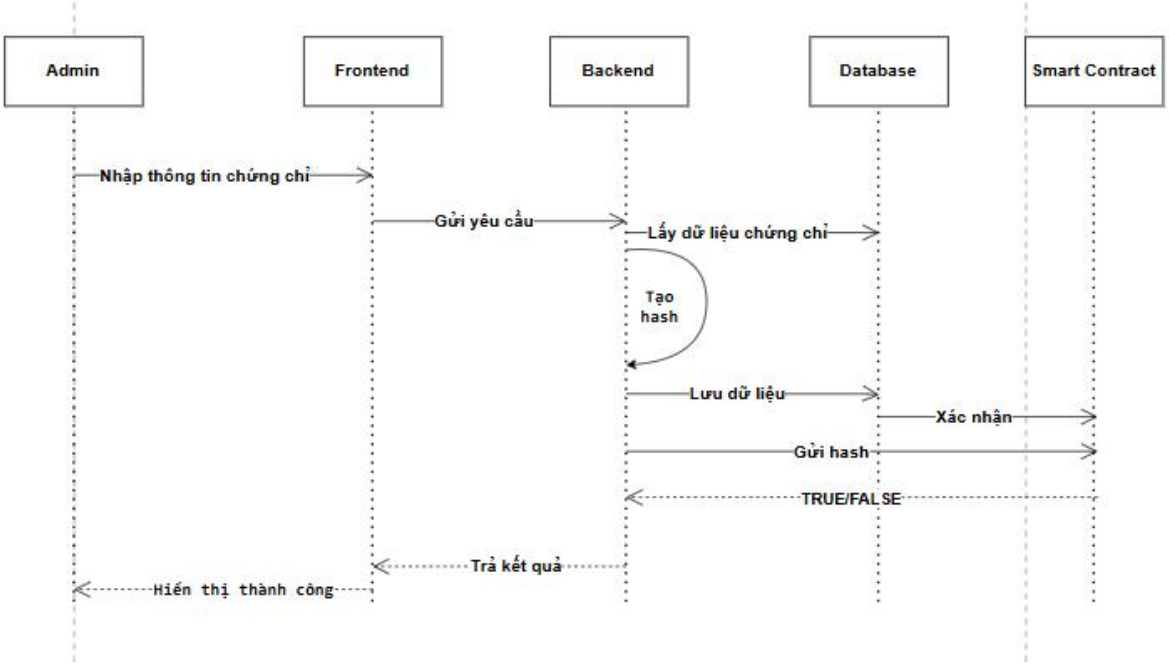
Frontend gửi txHash → Backend

Bước 8

Backend update ACTIVE



Luồng Issue Certificate



Sequence Diagram – Issue Certificate

4. YÊU CẦU CHỨC NĂNG (FUNCTIONAL REQUIREMENTS)

4.1 FR1 – Issue Certificate (Cấp chứng chỉ)

Mô tả

Hệ thống cho phép **Issuer** phát hành chứng chỉ cho sinh viên.
Quá trình này bao gồm việc lưu trữ dữ liệu off-chain (Database) và ghi nhận hash lên Blockchain nhằm đảm bảo tính bất biến.

Luồng xử lý chi tiết

1. Issuer nhập thông tin chứng chỉ trên giao diện Frontend, bao gồm:
    1. StudentID
    2. CourseName
    3. IssueDate
    4. MetadataURI (nếu có)
2. Frontend gửi yêu cầu đến Backend thông qua API.
3. Backend thực hiện:

    1. Kiểm tra tính hợp lệ dữ liệu
    2. Sinh giá trị **DataHash** từ nội dung chứng chỉ
    3. Lưu dữ liệu chứng chỉ vào Database với trạng thái PENDING

4. Frontend yêu cầu người dùng ký transaction thông qua MetaMask.
5. MetaMask thực hiện:

    1. Ký transaction bằng Private Key của Issuer
    2. Gửi transaction lên Blockchain để gọi hàm issueCertificate

6. Smart Contract:

    1. Lưu giá trị hash của chứng chỉ
    2. Ghi nhận chứng chỉ trên Blockchain

7. Sau khi transaction thành công:

    1. Frontend nhận txHash

8. Frontend gửi txHash về Backend
9. Backend cập nhật trạng thái chứng chỉ:

    1. Từ PENDING → ACTIVE
    2. Lưu BlockchainTxHash

Kết quả

- Chứng chỉ được phát hành thành công
- Dữ liệu được lưu ở cả Database và Blockchain

4.2 FR2 – View Certificate (Xem chứng chỉ)

Mô tả

Hệ thống cho phép **Holder**(Student) xem danh sách chứng chỉ của mình.

Thông tin hiển thị

- Tên sinh viên
- Tên khóa học
- Ngày cấp
- Tổ chức cấp (Issuer)
- Trạng thái chứng chỉ (PENDING / ACTIVE / REVOKED)

Luồng xử lý

1. Student truy cập giao diện xem chứng chỉ

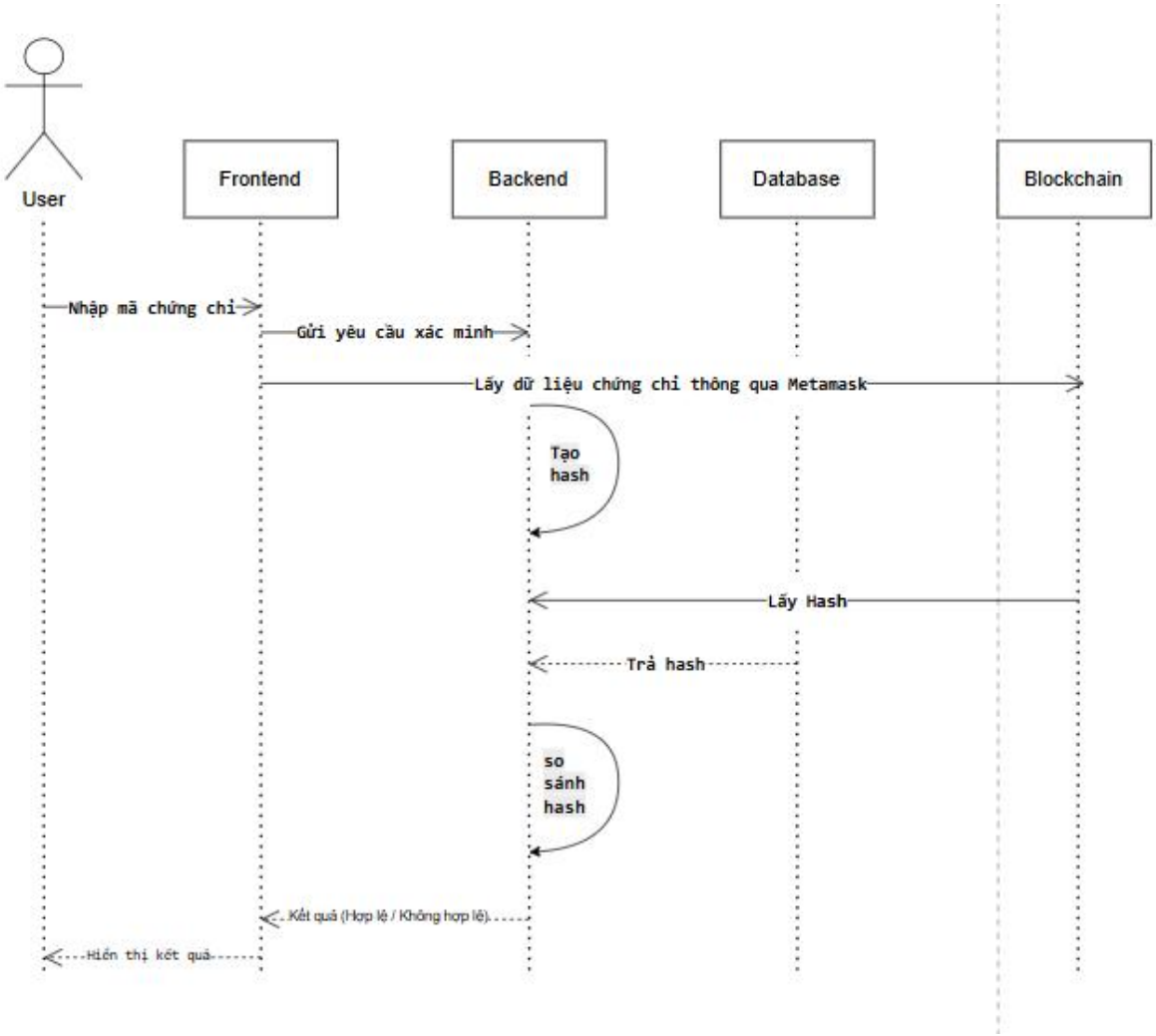
2. Frontend gửi request đến Backend
3. Backend truy vấn Database theo StudentID
4. Trả danh sách chứng chỉ về Frontend

4.3 FR3 – Verify Certificate (Xác minh chứng chỉ)

Mô tả

Hệ thống cho phép **Verifier** kiểm tra tính hợp lệ của chứng chỉ thông qua:

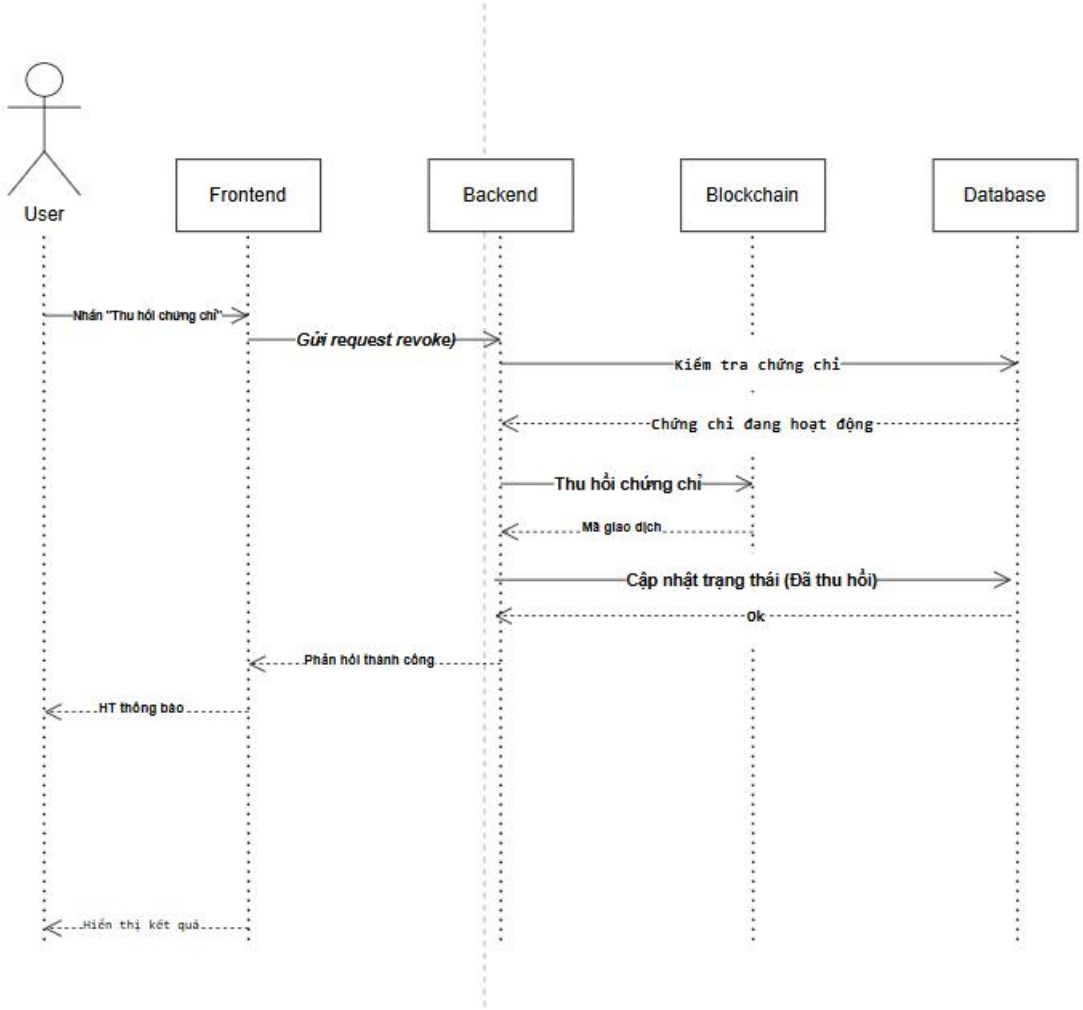
- CertificateID
- QR Code



FR3 – Verify Certificate

Luồng xử lý chi tiết

1. Verifier nhập CertificateID hoặc quét QR Code
2. Frontend gửi yêu cầu đến Backend
3. Backend thực hiện:
    - Lấy dữ liệu chứng chỉ từ Database
    - Tạo lại giá trị hash từ dữ liệu
4. Backend truy vấn Blockchain:
    - Lấy hash gốc đã lưu trong Smart Contract
5. Backend thực hiện so sánh:
    - Hash từ Database vs Hash trên Blockchain



Entity Relationship Diagram

6. Kết luận:

- Nếu hash trùng và trạng thái = ACTIVE → VALID
- Nếu hash trùng nhưng trạng thái = REVOKED → INVALID (đã thu hồi)
- Nếu hash không trùng → INVALIDBackend trả kết quả về Frontend

Kết quả trả về

- Trạng thái xác minh: VALID / INVALID
- Thông tin chứng chỉ

4.4 FR4 – Revoke Certificate (Thu hồi chứng chỉ)

Mô tả

Hệ thống cho phép **Issuer hoặc Admin** thu hồi chứng chỉ đã cấp.

Điều kiện

- Chỉ Issuer đã cấp hoặc Admin mới có quyền thu hồi

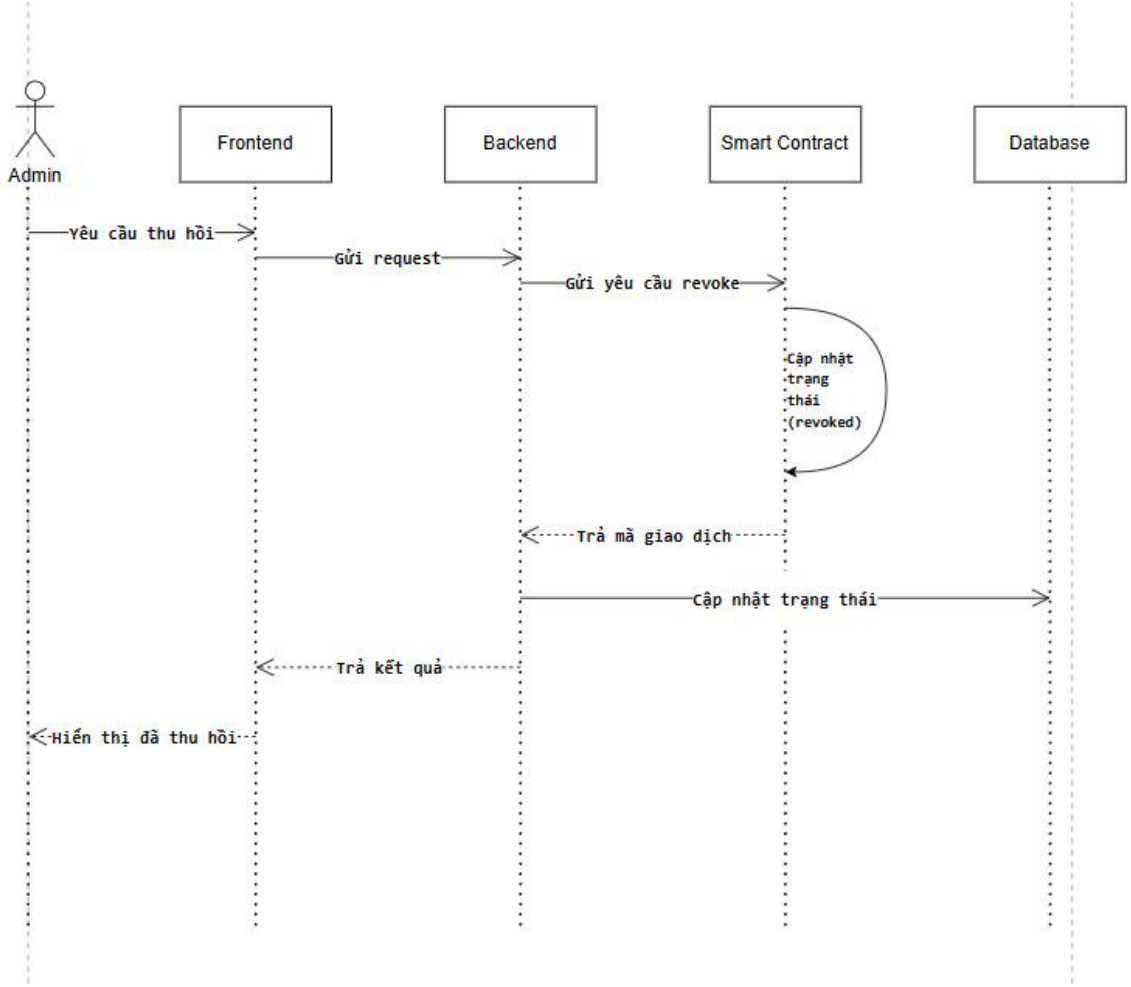
Luồng xử lý chi tiết

1. Issuer chọn chức năng "Revoke" trên giao diện
2. Frontend gửi yêu cầu kiểm tra đến Backend
3. Backend:
    - Kiểm tra trạng thái chứng chỉ (phải là ACTIVE)
    - Kiểm tra quyền của người thực hiện
4. Nếu hợp lệ:
    - Frontend yêu cầu MetaMask ký transaction
5. MetaMask:
    - Ký và gửi transaction gọi hàm revokeCertificate
6. Blockchain:
    - Đánh dấu chứng chỉ là đã thu hồi
7. Frontend nhận txHash
8. Frontend gửi txHash về Backend

9. Backend cập nhật Database:
    - Status → REVOKED

Kết quả

- Chứng chỉ bị thu hồi trên cả Blockchain và Database



FR4 – Revoke Certificate

4.5 FR5 – Generate QR Code (Tạo mã QR)

Mô tả

Mỗi chứng chỉ được cấp một mã QR phục vụ xác minh nhanh.

Luồng xử lý

1. Backend tạo URL xác minh:

/verify/{dataHash}

2. Frontend tạo QR Code từ URL
3. Người dùng quét QR để truy cập trang xác minh

5. THIẾT KẾ CƠ SỞ DỮ LIỆU (DATABASE DESIGN)

Hệ thống sử dụng **SQL Server** với các bảng chính sau:

5.1 Bảng Users

Lưu thông tin người dùng hệ thống:

- UserID (PK)
- WalletAddress
- Role (ADMIN / ISSUER / STUDENT)
- Name
- CreatedAt

5.2 Bảng Issuers

Lưu thông tin tổ chức cấp chứng chỉ:

- IssuerID (PK)
- UserID (FK)
- IsActive
- CreatedAt

5.3 Bảng Certificates

Lưu thông tin chứng chỉ:

- CertificateID
- StudentID (FK)
- IssuerID (FK)
- CourseName
- IssueDate
- DataHash
- BlockchainTxHash
- Status
- MetadataURI

- CreatedAt

5.4 Bảng CertificateLogs

Lưu lịch sử thao tác:

- LogID
- CertificateID
- Action (ISSUE / REVOKE / VERIFY)
- TxHash
- CreatedAt

6. SMART CONTRACT SPECIFICATION

Smart Contract được viết bằng **Solidity** và triển khai trên mạng Blockchain.

6.1 Các hàm chính

issueCertificate(hash)

- Lưu hash của chứng chỉ lên Blockchain

revokeCertificate(id)

- Đánh dấu chứng chỉ đã bị thu hồi

verifyCertificate(hash)

- Kiểm tra hash có tồn tại hay không

6.2 Nguyên tắc thiết kế

- Chỉ lưu hash, không lưu dữ liệu chi tiết
- Chỉ Issuer được phép cấp chứng chỉ
- Chỉ Issuer hoặc Admin được revoke
- Dữ liệu trên Blockchain là bất biến

7. HASH GENERATION

Thuật toán sử dụng

- SHA-256

Công thức

```
hash = SHA256(
  CertificateID +
  StudentID +
  CourseName +
  IssueDate +
  Issuer
)
```

Ý nghĩa

- Đảm bảo tính toàn vẹn dữ liệu
- Phát hiện thay đổi
- Ngăn chặn giả mạo

8. YÊU CẦU PHI CHỨC NĂNG (NON-FUNCTIONAL REQUIREMENTS)

NFR-01: Hiệu năng (Performance)

- Hệ thống phải phản hồi API trong thời gian ≤ 3 giây trong điều kiện bình thường
- Thời gian xác minh chứng chỉ (verify) ≤ 5 giây (do phụ thuộc Blockchain)

NFR-02: Bảo mật (Security)

Phân tách rõ 2 chế độ (đây là điểm "ăn tiền"):

Production (chuẩn DApp):

- Private Key KHÔNG được lưu trên Backend
- Tất cả giao dịch Blockchain phải được ký qua MetaMask

Development / Demo:

- Backend có thể sử dụng Private Key để tự ký transaction
- Chỉ dùng cho mục đích demo/test

NFR-03: Khả năng mở rộng (Scalability)

- Hệ thống hỗ trợ nhiều Issuer đồng thời
- Có thể mở rộng sang nhiều Smart Contract (multi-contract)
- Database hỗ trợ indexing để tối ưu truy vấn

NFR-04: Tính dễ sử dụng (Usability)

- Giao diện đơn giản, rõ ràng
- Hỗ trợ xác minh bằng QR Code
- Không yêu cầu kiến thức Blockchain đối với Verifier

NFR-05: Độ tin cậy (Reliability)

- Dữ liệu chứng chỉ không bị mất khi lỗi hệ thống
- Trạng thái Blockchain và Database phải đồng bộ

NFR-06: Khả năng bảo trì (Maintainability)

- Backend tách module rõ ràng (routes, services, models)
- Smart Contract có thể nâng cấp bằng deploy version mới
- Database thiết kế chuẩn hóa, dễ mở rộng

9. TRẠNG THÁI CHỨNG CHỈ

Trạng thái	Ý nghĩa
PENDING	Chưa ghi Blockchain
ACTIVE	Đã ghi Blockchain
REVOKED	Đã thu hồi
FAILED	Ghi Blockchain thất bại

10. CÔNG NGHỆ SỬ DỤNG

Frontend

- React
- ethers.js

Backend

- Node.js
- Express

Database

- SQL Server

Blockchain

- Solidity
- Sepolia / Ganache

Wallet

- MetaMask

11. QUY TẮC NGHIỆP VỤ (BUSINESS RULES)

BR-01

Mỗi chứng chỉ chỉ được cấp **một lần duy nhất** cho mỗi sinh viên và khóa học (trừ khi trạng thái trước đó là FAILED)

BR-02

Chỉ Issuer hoặc Admin mới có quyền:

- Cấp chứng chỉ
- Thu hồi chứng chỉ

BR-03

Chứng chỉ chỉ được xác minh là hợp lệ khi:

- Hash trên Database trùng với hash trên Blockchain
- Trạng thái là ACTIVE

BR-04

Chứng chỉ bị thu hồi (REVOKED) sẽ:

- Không còn hợp lệ khi verify
- Vẫn tồn tại trên Blockchain (không bị xóa)

BR-05

Backend chỉ cập nhật trạng thái ACTIVE khi:

- Transaction trên Blockchain thành công (có txHash hợp lệ)

12. USE CASE SPECIFICATION (TÙY CHỌN NHƯNG NÊN CÓ)

12.1 Tổng quan

Phần này mô tả chi tiết các Use Case chính của hệ thống nhằm làm rõ:

- Tương tác giữa Actor và hệ thống
- Luồng xử lý nghiệp vụ
- Điều kiện trước/sau

Các Use Case được thiết kế dựa trên các chức năng cốt lõi:

- Issue Certificate
- Verify Certificate
- Revoke Certificate
- View Certificate

12.2 Use Case UC-01: Issue Certificate

Tên: UC-01 – Cấp chứng chỉ
Actor: Issuer
Mô tả: Cho phép Issuer phát hành chứng chỉ cho sinh viên và ghi nhận lên Blockchain

Pre-condition

- Issuer đã đăng nhập hệ thống
- Ví MetaMask đã kết nối
- Issuer có quyền hợp lệ

Main Flow

1. Issuer nhập thông tin chứng chỉ (Student, CourseName, IssueDate, MetadataURI)
2. Frontend gửi request đến Backend
3. Backend:
    - Kiểm tra dữ liệu
    - Tạo DataHash
    - Lưu DB với trạng thái PENDING
4. Frontend yêu cầu MetaMask ký transaction
5. MetaMask gửi transaction lên Blockchain (issueCertificate)
6. Blockchain xác nhận giao dịch thành công
7. Frontend nhận txHash
8. Frontend gửi txHash về Backend
9. Backend cập nhật trạng thái ACTIVE

Post-condition

- Chứng chỉ tồn tại trên Blockchain
- Database đồng bộ trạng thái ACTIVE

12.3 Use Case UC-02: Verify Certificate

Tên: UC-02 – Xác minh chứng chỉ
Actor: Verifier / Student
Mô tả: Kiểm tra tính hợp lệ của chứng chỉ

Pre-condition

- Người dùng có CertificateID hoặc QR Code

Main Flow

1. Người dùng nhập ID hoặc quét QR
2. Frontend gửi request đến Backend
3. Backend:

    - Lấy dữ liệu từ Database
    - Tạo lại hash

4. Backend gọi Blockchain để lấy hash gốc
5. Backend so sánh:

    - Hash DB vs Blockchain
    - Trạng thái chứng chỉ

6. Trả kết quả về Frontend

Post-condition

- Trả về trạng thái:

    - VALID
    - INVALID

12.4 Use Case UC-03: Revoke Certificate

Tên: UC-03 – Thu hồi chứng chỉ
Actor: Issuer / Admin

Pre-condition

- Người dùng có quyền
- Chứng chỉ đang ở trạng thái ACTIVE

Main Flow

1. Issuer chọn chức năng Revoke
2. Frontend gửi request kiểm tra đến Backend
3. Backend kiểm tra quyền và trạng thái
4. Frontend yêu cầu MetaMask ký transaction
5. Blockchain thực hiện revokeCertificate
6. Frontend nhận txHash
7. Frontend gửi txHash về Backend
8. Backend cập nhật trạng thái REVOKED

Post-condition

- Chứng chỉ bị thu hồi trên Blockchain và Database

12.5 Use Case UC-04: View Certificate

Tên: UC-04 – Xem chứng chỉ
Actor: Student

Main Flow

1. Student truy cập trang chứng chỉ
2. Frontend gọi API Backend
3. Backend truy vấn Database
4. Trả danh sách chứng chỉ

Post-condition

- Hiển thị danh sách chứng chỉ

12.6 Use Case UC-05: Generate QR Code

Tên: UC-05 – Tạo QR Code
Actor: System

Main Flow

1. Backend tạo URL:
   /verify/{dataHash}
2. Frontend sinh QR Code
3. Người dùng sử dụng QR để verify

13. TRACEABILITY MATRIX

13.1 Mục đích

Traceability Matrix giúp:

- Liên kết yêu cầu → triển khai
- Đảm bảo không thiếu chức năng
- Hỗ trợ kiểm thử

13.2 Bảng Traceability

FR ID	Use Case	API Endpoint	Database	Smart Contract	UI
FR1 – Issue Certificate	UC-01	POST /api/certificates	Certificates	issueCertificate(hash)	Form Issue
FR2 – View Certificate	UC-04	GET /api/certificates	Certificates	-	Dashboard
FR3 – Verify Certificate	UC-02	GET /api/verify	Certificates	verifyCertificate(hash)	Verify Page
FR4 – Revoke Certificate	UC-03	PUT /api/certificates/revoke	Certificates	revokeCertificate(id)	Revoke Button
FR5 – Generate QR	UC-05	-	Certificates	-	QR Code

13.3 Phân tích

Tính đầy đủ

- Tất cả Functional Requirements đều có Use Case tương ứng
- Không có chức năng "mồ côi"

Tính nhất quán

- Mỗi Use Case đều:
    - Gắn với API
    - Gắn với DB
    - Gắn với Blockchain (nếu có)

Tính kiểm thử

- Có thể viết test case trực tiếp từ bảng này

14. TEST CASE SPECIFICATION

14.1 Mục đích

Phần này mô tả các test case nhằm:

- Kiểm chứng các Functional Requirements (FR)
- Đảm bảo hệ thống hoạt động đúng logic DApp
- Phát hiện lỗi ở cả **Backend – Database – Blockchain**

14.2 Phạm vi kiểm thử

Bao gồm:

- Issue Certificate
- Verify Certificate
- Revoke Certificate
- View Certificate
- QR Verification

14.3 Môi trường kiểm thử

Thành phần	Mô tả
Frontend	React
Backend	Node.js + Express
Database	SQL Server
Blockchain	Ganache / Sepolia
Wallet	MetaMask
RPC	Localhost / Infura

14.4 TEST CASE CHI TIẾT

TC-01 – Issue Certificate thành công

Thuộc tính	Giá trị
Test Case ID	TC-01

Thuộc tính	Giá trị
FR liên quan	FR1
Mô tả	Cấp chứng chỉ thành công
Actor	Issuer

Input

- StudentID hợp lệ
- CourseName hợp lệ
- IssueDate hợp lệ

Steps

1. Nhập dữ liệu trên UI
2. Click "Issue"
3. Ký MetaMask
4. Chờ transaction

Expected Result

- DB: status = ACTIVE
- Có BlockchainTxHash
- Blockchain có record hash

TC-02 – Issue Certificate thiếu dữ liệu

Thuộc tính	Giá trị
Test Case ID	TC-02
FR	FR1
Mô tả	Thiếu field bắt buộc

Input

- Missing StudentID

Expected

Backend trả lỗi:

- Thiếu dữ liệu bắt buộc
- Không ghi DB
- Không gọi blockchain

TC-03 – Issue Certificate nhưng MetaMask reject

Test Case ID	TC-03
Mô tả	User từ chối ký

Steps

1. Nhấn Issue
2. Reject MetaMask

Expected

- DB giữ trạng thái PENDING
- Không có txHash

TC-04 – Issue nhưng Blockchain fail

| Test Case ID | TC-04 |

Expected

- Status = FAILED
- Log lỗi được lưu

TC-05 – Verify Certificate hợp lệ

Test Case ID	TC-05
FR	FR3

Steps

1. Nhập CertificateID
2. Gửi verify

Expected

- Hash DB = Hash Blockchain
- Status = ACTIVE
  → VALID

TC-06 – Verify Certificate bị sửa dữ liệu

| Test Case ID | TC-06 |

Expected

- Hash mismatch
  → INVALID

TC-07 – Verify Certificate đã bị REVOKED

| Test Case ID | TC-07 |

Expected

- Hash đúng
- Status = REVOKED
  → INVALID

TC-08 – Verify bằng QR Code

| Test Case ID | TC-08 |

Steps

1. Scan QR
2. Mở URL verify

Expected

- Kết quả giống verify bằng ID

TC-09 – Revoke Certificate thành công

Test Case ID	**TC-09**
FR	FR4

Steps

1. Nhấn Revoke
2. Ký MetaMask

Expected

- Blockchain: revoked = true
- DB: status = REVOKED

TC-10 – Revoke không đủ quyền

| Test Case ID | TC-10 |

Expected

- Backend từ chối
- Không gọi blockchain

TC-11 – Revoke chứng chỉ không ACTIVE

| Test Case ID | TC-11 |

Expected

- Lỗi
- Certificate is not active

TC-12 – View Certificate

| Test Case ID | TC-12 |
| FR | FR2 |

Expected

- Trả danh sách đúng theo StudentID

TC-13 – Database mất kết nối

| Test Case ID | TC-13 |

Expected

- API trả lỗi 500
- Không crash server

TC-14 – RPC Node lỗi

| Test Case ID | TC-14 |

Expected

- Không gửi transaction
- Thông báo lỗi blockchain

TC-15 – Double Issue (trùng chứng chỉ)

| Test Case ID | TC-15 |

Expected

- Bị chặn bởi UNIQUE INDEX
- Không tạo record mới

14.5 PHÂN LOẠI TEST

Loại test	Áp dụng
Functional Test	TC-01 → TC-12
Negative Test	TC-02, TC-03, TC-10

Loại test	Áp dụng
Edge Case	TC-04, TC-11, TC-14
Integration Test	TC-01, TC-05, TC-09
Security Test	TC-10

14.6 KẾT LUẬN

Bộ test này đảm bảo:

- Bao phủ toàn bộ FR
- Kiểm tra đúng kiến trúc DApp:
    - MetaMask signing
    - Blockchain interaction
    - DB sync
- Có cả:
    - Success case
    - Failure case
    - Edge case

Danh Sách Thành Viên Nhóm 11

STT	MSSV	Họ tên	Nhiệm vụ
1	0023412629	Phạm Minh Chiến	● Phân tích thiết kế kiến trúc hệ thống (System Architecture). ● Xây dựng Smart Contract, triển khai trên Testnet (Sepolia). ● Phát triển Backend,xây dựng Core Backend API & Tích hợp Web3 (Ethers.js). ● Phát triển Frontend & Kết nối MetaMask. ● Kỹ thuật và logic hệ thống. ● Rà soát, chỉnh sửa và hỗ trợ
2	0023410186	Phạm Văn Tú	● Nghiên cứu cấu trúc dữ liệu và Database Schema. ● Xây dựng các module Backend cơ bản. ● Hỗ trợ soạn thảo tài liệu đặc tả kỹ thuật (SRS). ● Kiểm tra PPT
3	0023411535	Trần Nguyễn Phát Đạt	● Nghiên cứu giao diện người dùng (UI/UX). ● Xây dựng các thành phần (Components) tĩnh của ứng dụng. ● Chuẩn bị các sơ đồ
4	0023412001	Phan Thành Vinh	● Thiết kế slide thuyết trình (PowerPoint). ● Định dạng văn bản tài liệu đặc tả.